RÉSEAUX · FICHE DE RÉVISION · COURS 3

Couche réseau

Adressage IPv4, sous-réseaux, en-tête IP, fragmentation, IPv6, ICMP, routage, RIP

À RETENIR EN PRIORITÉ

- **Adresse réseau** = IP **AND** masque (partie hôte à 0) ; **broadcast** = partie hôte à 1 ; hôtes utilisables = $\mathbf{2^h - 2}$.
- **Classes** : A 0–127 (/8) · B 128–191 (/16) · C 192–223 (/24).
- **Sous-réseaux** : emprunter n bits à la partie hôte → 2^n sous-réseaux ; **taille de bloc = 256 − octet du masque**.
- **Fragmentation** : données par fragment = MTU − 20, arrondies au **multiple de 8** ; **offset = décalage en octets / 8** ; MF = 1 sauf pour le dernier.
- **IP** = sans connexion, au mieux, indépendant du support. **TTL** −1 par routeur. Protocole : 1 ICMP, 6 TCP, 17 UDP.
- **Choix de route** : distance administrative la plus faible (connecté 0, statique 1, OSPF 110, RIP 120), puis métrique la plus faible.

1. Rôle de la couche réseau et adresse IPv4

4 fonctions : adressage logique, encapsulation (segment → paquet), routage, désencapsulation. Protocoles : IP (v4, v6), ICMP, protocoles de routage (OSPF, RIP…).

IPv4 = **32 bits** = 4 octets écrits en décimal (0–255) séparés par des points. Elle se compose d'un **Network-ID** (le réseau) et d'un **Host-ID** (l'hôte, unique dans le réseau). Le **masque** les sépare : bits à **1** = réseau, bits à **0** = hôte. Configuration manuelle ou automatique (DHCP) ; voir avec `ipconfig` / `ifconfig`.

TUTO : DÉCIMAL ↔ BINAIRE (UN OCTET)

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Décimal → binaire : de gauche à droite, si la valeur tient, on met 1 et on la soustrait, sinon 0.
105 : 64 ✓ (reste 41), 32 ✓ (9), 16 ✗, 8 ✓ (1), 4 ✗, 2 ✗, 1 ✓ → `0110 1001`.

Binaire → décimal : additionner les valeurs des bits à 1.

VALEURS DE MASQUE À CONNAÎTRE

Bits à 1 dans l'octet	Valeur	Bloc (256 − valeur)
1	128	128
2	192	64
3	224	32
4	240	16
5	248	8
6	252	4
7	254	2
8	255	1

Ex. /26 = 8 + 8 + 8 + **2** → 255.255.255.**192**.

2. Classes, CIDR et adresses spéciales

Classe	1er octet	Masque par défaut	Bits réseau / hôte	Nb de réseaux	Hôtes par réseau
A	0 – 127	255.0.0.0 (/8)	8 / 24	2^7 = 128	2^{24} − 2 = 16 777 214
B	128 – 191	255.255.0.0 (/16)	16 / 16	2^{14} = 16 384	2^{16} − 2 = 65 534
C	192 – 223	255.255.255.0 (/24)	24 / 8	2^{21} = 2 097 152	2^8 − 2 = 254

CIDR : `/n` = nombre de bits à 1 du masque (partie réseau), sans notion de classe. Partie hôte = 32 − n bits.

- **Adresse réseau** : partie hôte à 0, 1re adresse de la plage.
- **Broadcast** : partie hôte à 1, dernière adresse de la plage.
- **127.0.0.1** (127.0.0.0/8) : bouclage (loopback), pour tester TCP/IP sur soi-même.
- **Privées** : réseau local uniquement, **non routables**, réutilisables d'un réseau à l'autre.
- **Publiques** : routables sur Internet, acheminées entre FAI.
- *Bon à savoir (hors diapo)* : plages privées 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

TUTO : ADRESSE RÉSEAU, BROADCAST, PLAGE ET NOMBRE D'HÔTES

1. Trouver le **masque** (classe ou /n) et l'**octet « intéressant »** : celui où le masque n'est ni 255 ni 0.
2. **Taille de bloc** = 256 − valeur du masque dans cet octet.
3. Lister les multiples du bloc (0, bloc, 2 × bloc…) : l'**adresse réseau** est le multiple juste ≤ à l'octet de l'IP ; les octets suivants passent à 0.
4. **Broadcast** = multiple suivant − 1 ; les octets suivants passent à 255.
5. **Plage utilisable** = réseau + 1 → broadcast − 1. **Hôtes** = $2^h - 2$ (h = 32 − n).

192.44.10.105 /26 (exemple du cours)
Masque 255.255.255.192 → octet 4, bloc = 256 − 192 = **64**
Multiples : 0, **64**, 128… → 105 est entre 64 et 128
Réseau **192.44.10.64** · broadcast **192.44.10.127**
Plage .65 → .126 · hôtes $2^6 - 2$ = **62**
`Vérif AND : 01101001 AND 11000000 = 01000000 = 64`

172.10.10.172 avec le masque par défaut
172 → classe B → /16 : les deux derniers octets sont la partie hôte
Réseau **172.10.0.0** · broadcast **172.10.255.255**

192.168.5.130 /27
Bloc = 256 − 224 = 32 → multiples 96, **128**, 160
Réseau **.128** · broadcast **.159** · 30 hôtes

3. Créer des sous-réseaux

On **emprunte des bits à la partie hôte** pour coder l'ID de sous-réseau : <réseau><sous-réseau><hôte>, et on les met à 1 dans le masque.

TUTO A : ON CONNAÎT LE NOMBRE DE SOUS-RÉSEAUX N

1. Plus petit n tel que $\mathbf{2^n \geq N}$.
2. Nouveau masque = masque de départ + n bits.
3. Bloc = 256 − octet du masque → adresses des sous-réseaux = multiples du bloc.
4. Pour chacun : 1re adresse + 1, broadcast = suivant − 1.

183.1.0.0 (classe B, /16), 4 sous-réseaux
$2^2 = 4$ → n = 2 → **/18 = 255.255.192.0**, bloc 64 (3e octet)

Sous-réseau	Plage	Broadcast
183.1.0.0	.0.1 → .63.254	183.1.63.255
183.1.64.0	.64.1 → .127.254	183.1.127.255
183.1.128.0	.128.1 → .191.254	183.1.191.255
183.1.192.0	.192.1 → .255.254	183.1.255.255

TUTO B : ON CONNAÎT LE NOMBRE D'HÔTES H PAR SOUS-RÉSEAU

1. Plus petit h tel que $\mathbf{2^h - 2 \geq H}$ (bits pour l'hôte).
2. Masque = **/(32 − h)**.
3. Bits de sous-réseau = nouveau masque − masque de départ → $\mathbf{2^{(bits)}}$ **sous-réseaux**.
4. Bloc, adresses et broadcasts comme au tuto A.

128.99.0.0 (classe B), 1000 hôtes par sous-réseau
$2^{10} - 2 = 1022 \geq 1000$ → h = 10 → /22 = **255.255.252.0**
Sous-réseau : 22 − 16 = 6 bits → $\mathbf{2^6}$ **= 64 sous-réseaux**
Bloc = 256 − 252 = 4 (3e octet)
SR1 : **128.99.0.0** → broadcast **128.99.3.255**
SR2 : **128.99.4.0** → broadcast **128.99.7.255**

4. Le protocole IPv4

Caractéristiques : **sans connexion** (aucun contact préalable, envoi même si la destination ne peut pas recevoir) · **au mieux** (aucune garantie de livraison ni d'absence d'erreur ; la fiabilité vient de TCP) · **indépendant du support** (fibre, satellite, sans-fil… ; adapte la taille des paquets).

Champ	Bits	À retenir
Version	4	4 (`0100`) ou 6 (`0110`)
IHL	4	Longueur d'en-tête en **mots de 32 bits** : 5 → 20 octets (min) ; 15 → 60 octets (max). Octets = IHL × 4
Services différenciés	8	DSCP (6, priorité QoS : 000000 best effort, 101110 VoIP) + ECN (2, congestion : 11 = congestion signalée)
Total Length	16	En-tête + données en octets : 20 à 65 535
Identification	16	Identique pour tous les fragments d'un même paquet
Flags	3	0 (inutilisé) · **DF** interdit la fragmentation · **MF** = 1 si d'autres fragments suivent
Fragment Offset	13	Position du fragment, en **blocs de 8 octets**
TTL	8	−1 à chaque routeur ; à 0 le paquet est détruit (évite les boucles)
Protocole	8	**1 ICMP · 6 TCP · 17 UDP**
Header Checksum	16	Contrôle de l'**en-tête seulement**
IP source / destination	32 + 32	Puis options (0 à 40 octets) et données

5. La fragmentation

Chaque réseau a une **MTU** (taille max d'un paquet) : Ethernet **1500**, FDDI 4352, Token Ring 4440 à 17 940, X25 1007. Paquet > MTU → découpé en fragments. Les fragments gardent la même version, IHL, ToS, **identification**, TTL, protocole et adresses ; changent : **longueur, MF, offset, checksum**.

TUTO : FRAGMENTER UN PAQUET

1. **Données à transporter** = taille du paquet − 20 (en-tête).
2. **Charge max par fragment** = MTU − 20, **arrondie au multiple de 8 inférieur**.
3. **Nombre de fragments** = données / charge, arrondi **au-dessus**.
4. **Dernier fragment** = données − (nombre − 1) × charge.
5. **Offset du fragment k** (k = 0, 1, 2…) = k × charge / 8.
6. Tous : même Identification ; **MF = 1** sauf le dernier (MF = 0). Taille d'un fragment = ses données + 20.

Cours : paquet de 1500 octets, MTU 525
Données 1500 − 20 = 1480
Charge 525 − 20 = 505 → **504** (504 = 63 × 8)
1480 / 504 ≈ 2,9 → **3 fragments** : 504, 504, 472
Offsets : 0, 504/8 = **63**, 1008/8 = **126**
Flags : 001, 001, 000

Exercice : paquet de 576 octets, MTU 150
Données 576 − 20 = 556
Charge 150 − 20 = 130 → **128** ; fragment de 148 octets
556 / 128 ≈ 4,3 → **5 fragments**
Dernier : 556 − 4 × 128 = **44 octets**
Offsets : **0, 16, 32, 48, 64**

PIÈGES
Oublier de retirer les 20 octets d'en-tête ; donner l'offset en octets au lieu de blocs de 8 ; oublier d'arrondir la charge au multiple de 8.

6. IPv6

- **128 bits** (≈ 340 × 10^{36} adresses) contre 32 bits (≈ 4,3 milliards).
- Écriture : **8 blocs de 16 bits en hexa** séparés par « : ».
- Apports : en-tête simplifié, auto-configuration, **multicast au lieu du broadcast**, **IPsec** obligatoire, plus besoin de **NAT**.
- Coexistence avec IPv4 : **double pile**, **tunneling**, **NAT64**.
- En-tête de taille **fixe : 40 octets**.

IPv4	IPv6
Version (4)	Version (6)
Services différenciés	Traffic Class
—	**Flow Label** (20 bits, nouveau : flux à traiter spécialement, ex. VoIP)
Total Length	Payload Length (sans l'en-tête de base)
Protocole	Next Header (6 TCP, 17 UDP, 58 ICMPv6)
TTL	Hop Limit
IP src / dest 32 bits	IP src / dest 128 bits

TUTO : ABRÉGER UNE ADRESSE IPV6

1. Retirer les zéros en tête de chaque bloc. 2. Remplacer **une seule** suite de blocs nuls par `::`.

`2001:0db8:0000:0000:0101:abcd:0000:1234` → `2001:db8:0:0:101:abcd:0:1234` → `2001:db8::101:abcd:0:1234`

7. ICMP

ICMP (couche 3) contrôle les erreurs de transmission ; il est encapsulé dans IP (**Protocole = 1**). `ping` envoie des **requêtes d'écho (type 8)** et reçoit des **réponses d'écho (type 0)**.

Type	Signification
0 / 8	Réponse / requête d'écho (ping)
3	Destination inaccessible (code 0 réseau, 1 hôte, 2 protocole, 3 port, 4 fragmentation nécessaire mais DF)
5	Redirection
11	Temps dépassé (TTL à 0 ou réassemblage trop long)
12	Problème de paramètre (en-tête erroné)

8. Le routage

Routage = trouver le meilleur chemin. **Passerelle par défaut** = l'équipement (routeur) qui fait sortir le trafic du réseau local. **Table de routage** : destination, masque, passerelle (« on-link » = joignable directement), interface, **métrique** (coût : la plus faible est préférée). `0.0.0.0/0` = **route par défaut**.

TUTO : OÙ L'HÔTE ENVOIE-T-IL LE PAQUET ?

1. Calculer le réseau de la destination (IP dest AND masque de chaque route).
2. Si c'est **son propre réseau** (route « on-link ») → envoi **direct** par son interface.
3. Sinon, aucune route précise → **route par défaut 0.0.0.0** → envoi à la **passerelle**, qui choisit la suite.

PC1 = 192.168.10.10/24, passerelle 192.168.10.1.
→ 192.168.10.20 : direct (on-link).
→ 10.10.10.10 : passerelle 192.168.10.1.

TUTO : QUELLE ROUTE LE ROUTEUR CHOISIT-IL ?

1. Même destination apprise par plusieurs sources → la plus petite **distance administrative**.
2. Même protocole → la plus petite **métrique**.

Origine	DA	Origine	DA
Connecté	0	IGRP	100
Statique	1	OSPF	110
Récap. EIGRP	5	IS-IS	115
BGP externe	20	RIP	120
EIGRP interne	90	EIGRP externe	170
		BGP interne	200

	Statique (configuré à la main, `ip route`)	Dynamique (protocoles de routage)
+	Simple, sécurisé, pas de trafic de mise à jour	Mise à jour automatique, pas de charge d'administration, évolutif
–	Mise à jour manuelle à chaque changement, erreurs, peu évolutif	Trafic de mises à jour ; OSPF et EIGRP demandent du processeur

Vecteur de distance : chaque route = distance (métrique) + direction (prochain saut). RIPv1, RIPv2, IGRP (obsolète), EIGRP.

État de lien : chaque routeur a la topologie complète ; mise à jour seulement en cas de changement. **OSPF**, **IS-IS**.

RIP

Vecteur de distance, métrique = **nombre de sauts**, **15 max** (16 = injoignable), mises à jour toutes les **30 s**.
Limites : petits réseaux, convergence lente, boucles.
RIPv1 : pas de VLSM, broadcast, pas d'authentification.
RIPv2 : VLSM, multicast **224.0.0.9**, authentification.
RIPng : IPv6.